CHAPTER 1

Advanced SQL and Database Management

If you recall from our basic SQL chapters, we have the ability to create sets of rows and columns that represent data. We can represent a set of 1000 bikes, each with a respective price, status, brand, and id, in the form of rows (you may hear the term tuples) and columns (attributes). In this chapter, we will build out a larger database, with more than one table, but, here's the twist: we will find a way to link them together, so that data can be searched across tables.

1.1 Keys

In the experimentation from our first chapter, you may have seen the term *primary key*. An attribute (or set of attributes) that can uniquely identify rows within a table are called a primary key. That may have just sounded like a set of buzzwords, so let's take a step back:

Take for example the following table:

StudentID	**Email**	**SSN**	FirstName	LastName	EnrollmentDate
1001	alice@univ.edu	123-45-6789	Alice	Smith	2024-01-15
1002	bob@univ.edu	987-65-4321	Bob	Johnson	2024-01-15
1003	carol@univ.edu	555-55-5555	Carol	Williams	2024-02-01

Table 1.1: STUDENT table demonstrating key types.

We can identify columns or combinations of columns that could be used to uniquely identify each row. We can identify pretty much infinite *super keys*, which are any set of attributes that uniquely identifies a row. What sets these apart is that they may include extraneous attributes that are not required for uniqueness. For example, in the STUDENT table, both {StudentID} and {StudentID, Email} are super keys, but the latter includes an unnecessary attribute for uniqueness. Keys that contain more than one attribute are called *composite keys*, and are represented in sequence with parentheses such as (StudentID, Email).

A *candidate key* is a minimal super key, meaning it has no extraneous attributes. In our STUDENT table, {StudentID} and {SSN} are candidate keys because they uniquely iden-

tify each row and do not contain unnecessary attributes.

From the candidate keys, a *primary key* is selected. By convention, the primary key is underlined. The primary key is the main candidate key chosen to uniquely identify each row in the table. In our STUDENT table, {StudentID} could be chosen as the primary key. Then, {SSN} would be considered an *alternate key*, which is a candidate key not selected as the primary key. The reason we select {StudentID} over {SSN} is that {SSN} has real-world representation, which we call a *natural key*. We do not want to conflate real world attributes with database management tools such as primary keys.

A real world example: What if we were using an email as the primary key? Then, if the email schema changes due to the business being bought out or renamed (this happened to one of our authors!), we are stuck! A change like <firstinitial><lastname>@company.com to <firstname>.<lastname>@company2.com would break the database.

This is why we introduce *surrogate keys*. Surrogate keys are keys with no real world meaning, often stable identifiers that do not change. This includes auto-incrementing integers, Universally Unique Identifiers (*UUIDs*), or Hex Codes that do not have the opportunity to change. In short, we prefer surrogate keys for primary keys to ensure stability and avoid issues with real-world attribute changes.

In a few sections we will explore how foreign keys allow us to link tables together using these primary keys.

1.2 Normalization

With all of these key types, our tables can get pretty large. When this happens, we will want to normalize our database to ensure efficiency and reduce **redundancy**.

We have a process called *normalization*, which involves organizing, reordering, and restructuring the columns and tables of a database to minimize redundancy and dependency. This can help

- improve performance by reducing the amount of redundant data stored, as databases reach high volumes of data.
- prevent anomalies, which are inconsistencies or errors that can occur when redundant data is updated, inserted, or deleted.
- maintain data integrity

Ultimately, there is one primary goal of normalization:

> The table can only express one version of truth.

1.2.1 Functional Dependencies

Normalization rests on a single formal idea: the *functional dependency*. We say an attribute (or set of attributes) Y is functionally dependent on attribute X, written $X \rightarrow Y$, if knowing the value of X tells us exactly one corresponding value of Y. In other words, X *determines* Y.

For example, in our STUDENT table from earlier, {StudentID} $\rightarrow$ {Email}, since each StudentID corresponds to exactly one Email. This is exactly why {StudentID} can serve as a key: a candidate key is a minimal set of attributes that functionally determines every other attribute in the table.

Two particular kinds of functional dependency will help us reason through the normal forms:

- A *partial dependency* occurs when a non-key attribute depends on only part of a composite primary key, rather than the whole key.
- A *transitive dependency* occurs when a non-key attribute depends on another non-key attribute, rather than depending directly on the primary key.

Removing partial dependencies brings a table to 2NF, and removing transitive dependencies brings a table to 3NF.

Let's go through an example. In this book, we will only focus on the first three normal forms: 1NF, 2NF, and 3NF. There are videos in your digital resources that go deeper into the higher normal forms.

Below is table representing an intentionally poorly designed database representing lab equipment at a university laboratory.

Table 1.2: Equipment Checkout Log

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42, E55	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer, Oscilloscope	Lab B, Lab A	Maria, David
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria

1

1.2.2 1NF

A database is in first normal form (1NF) if it meets the following criteria:

[1] Notice how small the text had to get to render the entire table within the page width, this is also a clear indication that something is off!

- Each column contains atomic values (no repeating groups or arrays).
- Each row is unique (no duplicate rows).

Notice how the table violates the first normal form (1NF) because it contains repeating groups and multiple values in a single column. In 1NF, each column should contain atomic values, and each row should be unique. Invalid attribute values include:

- Multiple `EquipmentIDs` in a single cell ({E42, E55})
- Multiple `EquipmentNames` in a single cell ({3D Printer, Oscilloscope})
- Multiple `EquipmentLocations` in a single cell ({Lab B, Lab A})
- Multiple `LocationManagers` in a single cell ({Maria, David})

To remedy this, we make a second row for student Alex: Now our table conforms to the

Table 1.3: Equipment Checkout Log [1NF]

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer	Lab B	Maria
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria
1001	E55	2026-09-22	Alex	alex@univ.edu	Junior	Oscilloscope	Lab A	David

first normal form (1NF), with each column containing atomic values and each row being unique.

1.2.3 2NF

1NF was quite easy, now we move on to the second normal form (2NF), which requires the following:

- The table is in 1NF.
- No partial dependencies (no non-key attribute should depend on a part of a composite primary key).
- All non-key attributes are fully functionally dependent on the primary key.

For demonstration purposes, lets add a fourth entry into our table:

Table 1.4: Equipment Checkout Log [Pre-2NF]

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer	Lab B	Maria
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria
1001	E55	2026-09-22	Alex	alex@univ.edu	Junior	Oscilloscope	Lab A	David
1001	E74	2026-09-23	Alex	alex@univ.edu	Junior	CNC Machine	Lab C	Larry

We can establish that the primary key of this table is the combination of {`StudentID`, `EquipmentID`, `CheckoutDate`}, as each student can check out multiple pieces of equipment, and each piece of equipment can be checked out by multiple students.

However, we have lots of repeated data in here:

- Student information (name, email, year) is repeated for each piece of equipment they check out.
- Equipment information (name, location, location manager) is repeated for each student who checks it out.

This redundancy indicates that our table is not in 2NF, as non-key attributes are partially dependent on the composite primary key. To achieve 2NF, we need to separate the student and equipment information into their own tables, leaving only the checkout details in the original table.

The new schemas will look like this:

```
Student (StudentID, StudentName, StudentEmail, StudentYear)
Equipment (EquipmentID, EquipmentName, EquipmentLocation, LocationManager)
Checkout (StudentID, EquipmentID, CheckoutDate)
```

StudentID	StudentName	StudentEmail	StudentYear
1001	Alex	alex@univ.edu	Junior
1002	Jordan	jordan@univ.edu	Senior

Table 1.5: Student Table [2NF]

EquipmentID	EquipmentName	EquipmentLocation	LocationManager
E42	3D Printer	Lab B	Maria
E55	Oscilloscope	Lab A	David
E74	CNC Machine	Lab C	Larry

Table 1.6: Equipment Table [2NF]

StudentID	EquipmentID	CheckoutDate
1001	E42	2026-09-20
1002	E42	2026-09-21
1001	E55	2026-09-22
1001	E74	2026-09-23

Table 1.7: Checkout Table [2NF]

Notice how each table now has a single, clear primary key: {StudentID} for Student, {EquipmentID} for Equipment, and {StudentID, EquipmentID} for Checkout. Every non-key attribute is now fully dependent on its table's key, with no partial dependencies remaining. 2NF often splits up a table into multiple tables to remove partial dependencies and serves as a stepping stone to 3NF, which further eliminates transitive dependencies.

1.2.4 3NF

To transform a table from 2NF to 3NF, we need to meet the following conditions:

- The table must already be in 2NF (and, consequently, in 1NF).
- There should be no *transitive dependencies*, meaning non-key attributes should not depend on other non-key attributes.

Looking at our table, we see a transitive dependency:

$$\texttt{EquipmentID} \rightarrow \texttt{EquipmentLocation} \rightarrow \texttt{LocationManager}$$

These are all stored in the same table, which creates a transitive dependency that violates 3NF.

We now must make a new table to remove this transitive dependency, ensuring that each non-key attribute depends only on the primary key of its table.

```
Equipment (EquipmentID, EquipmentName, EquipmentLocation)
Location (EquipmentLocation, LocationManager)
```

EquipmentID	EquipmentName	EquipmentLocation
E42	3D Printer	Lab B
E55	Oscilloscope	Lab A
E74	CNC Machine	Lab C

Table 1.8: Equipment Table [3NF]

EquipmentLocation	LocationManager
Lab B	Maria
Lab A	David
Lab C	Larry

Table 1.9: Location Table [3NF]

FIXME exercise transform a full table up to 3NF

1.3 Entity-Relationship Diagram

Now that we have normalized our tables, we can create an *entity-relationship diagram* to visually represent the relationships between the tables. Often, this process may even clear up further FDs or issues with the database design.

To do this, we will introduce a new programming language called *Mermaid*, which allows us to create diagrams directly within LaTeXdocuments. It is a text-based diagramming tool that is universally used in other software that supports Mermaid syntax, such as GitHub, Obsidian, and Notion.

Using our normalized tables, we can create an ER diagram to visualize the relationships between them.

We can say that our `EquipmentLocation` requires having a `LocationManager`, and (for the sake of our database), each `LocationManager` is responsible for managing a unique location. Therefore, we can say that `LocationManager` and `EquipmentLocation` have a *one-to-one relationship*.

We have added the mermaid diagram in Figure 1.1a, but here is the mermaid syntax for it:

```
erDiagram
    EQUIPMENT_LOCATION ||--|| LOCATION_MANAGER : "has (one to one)"
```

How about when it comes to a Lab Location and Lab Equipment? Well, a Location that is recently built could have zero pieces of equipment associated with it, while an existing location might have one or more pieces of equipment. However, a given piece of equipment will always be associated with exactly one location, even if there are multiple of that piece of equipment in other locations (think: multiple 3D printers in different labs).

This scenario represents a *zero-to-many relationship* between the `Location` and `Equipment` tables (see Figure 1.1b).

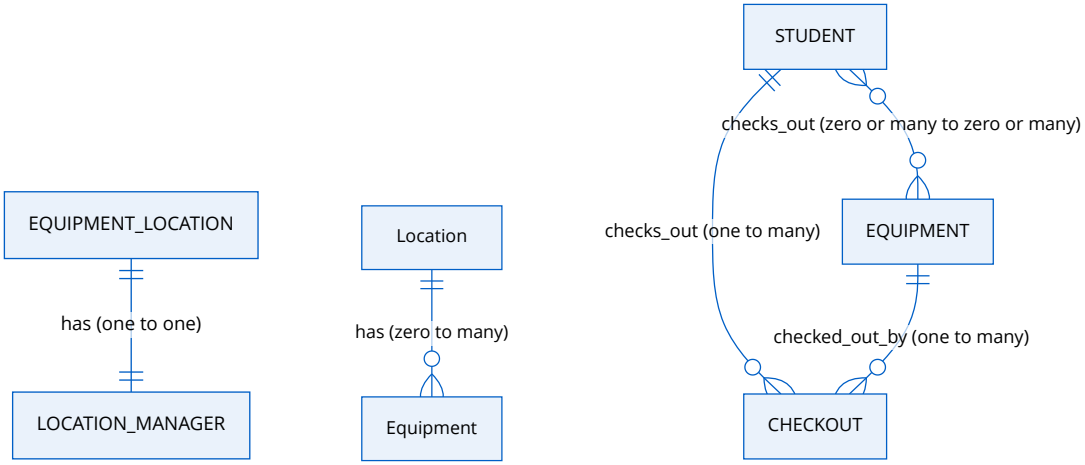
```
erDiagram
    Location ||--o{ Equipment : "has (zero to many)"
```

A student can check out many pieces of equipment, and a piece of equipment can be checked out by many students (see Figure 1.1c). This is the relationship behind the whole Checkout table from the 2NF split. We did this so that we could accurately track which student has checked out which piece (or pieces of equipment).

```
erDiagram
```

```
    STUDENT }o--o{ EQUIPMENT : checks_out
    STUDENT ||--o{ CHECKOUT : "checks_out (one to many)"
    EQUIPMENT ||--o{ CHECKOUT : "checked_out_by (one to many)"

```



(a) One-to-one (b) Zero-to-many (c) Many-to-many, resolved through `Checkout`

Figure 1.1: ER diagrams for the three relationship types in the lab equipment database.

1.4 Relational Algebra Basics

Relational Algebra	SQL
π	`SELECT`
$\times$	, or `CROSS JOIN`
$\bowtie_\theta$	`INNER JOIN <table> ON <condition>`
σ	`WHERE`
τ	`ORDER BY`
$\cup$	`UNION`
$-$	`EXCEPT`
$\cap$	`INTERSECT`

1.5 Relating Two Tables

Now let's say we have two tables, `Equipment` and `Location`, and we want to relate them using a *foreign key*. The `Equipment` table has a column `EquipmentLocation` that references the primary key `EquipmentLocation` in the `Location` table. This allows us to link each piece of equipment to its corresponding location.

Let's first create the `Equipment` table with a foreign key referencing the `Location` table.

```sql
CREATE TABLE Location (
    EquipmentLocation VARCHAR(50) PRIMARY KEY,
    LocationManager VARCHAR(50) UNIQUE
);

CREATE TABLE Equipment (
    EquipmentID VARCHAR(50) PRIMARY KEY,
    EquipmentName VARCHAR(50),
    EquipmentLocation VARCHAR(50) NOT NULL, -- the not null constraint ensures
    ↪ that every piece of equipment must have a location
    FOREIGN KEY (EquipmentLocation) REFERENCES Location(EquipmentLocation)
);
```

We *constrain* the values in the `EquipmentLocation` column of the `Equipment` table to only those that exist in the `Location` table, ensuring referential integrity between the two tables.

Using relational algebra, we can say that

$$\pi_{\text{EquipmentLocation}}(\text{Equipment}) \subseteq \pi_{\text{EquipmentLocation}}(\text{Location})$$

All values in the `EquipmentLocation` column of the `Equipment` table must exist in the `EquipmentLocation` column of the `Location` table. However, it does not guarantee that every location in the `Location` table has corresponding equipment in the `Equipment` table. For example, here are the two tables again, with altered data

EquipmentLocation	LocationManager
Lab B	Maria
Lab A	David
Lab C	Larry

Table 1.10: Location Table

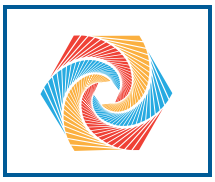
EquipmentID	EquipmentName	EquipmentLocation
E42	3D Printer	Lab B
E55	Oscilloscope	Lab A

Table 1.11: Equipment Table

Table constraints do not break when foreign key values are missing in the referencing table. For example, even though "Lab C" exists in the `Location` table, it does not appear in the `Equipment` table, and this is perfectly valid. However, if we tried to insert a row into the `Equipment` table with an `EquipmentLocation` value that does not exist in the `Location` table (such as "Lab D"), the database would reject it due to the foreign key constraint.

Appendix A

Answers to Exercises



Index

alternate key, 2

candidate key, 1
composite keys, 1

determines, 3

entity-relationship diagram, 7
ER diagram, *see* entity-relationship diagram

foreign key, 8
functional dependency, 3

Mermaid, 7

natural key, 2
normalization, 2

one-to-one relationship, 7

partial dependency, 3
primary key, 1, 2

super keys, 1
surrogate keys, 2

transitive dependencies, 6
transitive dependency, 3

UUIDs, 2

zero-to-many relationship, 7